# V-OS

Un système d'exploitation graphique pour l'Amstrad CPC 6128

Manuel

Fait à partir de docs/manual/fr/*.md et sdk/README.fr.md par tools/mkdocs.py (make docs).

# Sommaire

V-OS est un système d'exploitation graphique pour l'Amstrad CPC 6128, écrit entièrement en assembleur Z80. Il offre :

- une barre de menus, des fenêtres déplaçables, des dialogues et des boutons, utilisables au clavier ou avec une souris AMX
- trois vues pour les fichiers et les programmes (Bureau, Lanceur, Commander) et les réglages (Réglages)
- six langues sous forme de modules : anglais, grec, français, espagnol, allemand et italien
- des programmes qui tournent côte à côte et reprennent là où ils s'étaient arrêtés, un accessoire de bureau et des programmes en arrière-plan
- Studio, un éditeur de blocs à la Scratch qui tourne sur le CPC lui-même
- une API de 42 appels et un SDK pour les programmes en assembleur

Ce manuel existe aussi en grec : 00-index.md, en anglais : 00-index.md, en espagnol : 00-index.md, en allemand : 00-index.md et en italien : 00-index.md.


- 1. Premiers pas
- 2. Les vues
- 3. Les programmes
- 4. Les langues
- 5. Studio : des programmes avec des blocs
- 6. Les blocs sur le PC : l'éditeur web et `vbc.py`
- 7. Le SDK de V-OS : des programmes en assembleur
- 8. Développer V-OS

# 1. Premiers pas

## Ce qu'il vous faut

- Un Amstrad CPC 6128 ou un émulateur. V-OS est testé sur Caprice32.
- La disquette de V-OS dans le lecteur A:, c'est-à-dire le `build/vos.dsk` que produit `make`.
- En option :
  - une souris AMX
  - un second lecteur (B:)
  - une extension mémoire (par ex. 512K), pour plus de programmes à la fois

## Démarrage

Depuis le BASIC :

```
RUN"VOS
```

V-OS charge le noyau, coupe les ROM et prend le contrôle de toute la machine. Il détecte l'extension mémoire tout seul.

- **Première vue :** le Bureau. Vous pouvez la changer dans les Réglages, à la ligne VUE AU DÉMARRAGE.
- **Langue :** l'anglais. S'il y a un `SETUP.CFG` sur la disquette, V-OS démarre avec la langue et les réglages qui y sont enregistrés.
- **Sortie :** FICHIER > QUITTER VERS LE BASIC (CONTROL+Q dans les vues). Une confirmation est demandée, puis le CPC redémarre.

## L'écran

V-OS utilise tout l'écran (overscan) : 384×272 pixels en mode 1 (4 couleurs) ou 192×272 en mode 0 (16 couleurs).

- **Barre de menus** (en haut) : les menus de la vue ou du programme, en général FICHIER ou le menu du programme, AFFICHAGE et AIDE. À droite, la langue du clavier (EN, EΛ, FR, ...).
- **Ligne du bas :** dans chaque vue, les touches qui y fonctionnent.

## Les touches valables partout

| Touche | Ce qu'elle fait |
|---|---|
| F1 | Bureau |
| F2 | Lanceur |
| F3 | Commander |
| F4 | Réglages |
| F5 | ouvre et ferme l'accessoire de bureau (ACCESSOIRE) |
| F6 | les programmes en cours, pour en choisir un |
| ESC | ouvre le premier menu. Dans les menus, ←/→ changent de menu, ↑/↓ choisissent un élément, RETURN l'exécute, ESC ferme |
| CONTROL+lettre | les raccourcis des menus (par ex. ^Q, ^L) |

| | |
|---|---|
| CONTROL+L | changer de langue (AFFICHAGE > CHANGER DE LANGUE) |
| CONTROL+K | le dialogue des touches (AIDE > TOUCHES) |
| CONTROL+SHIFT | bascule le clavier entre l'anglais et la langue du module |
| TAB, SHIFT+TAB | déplacent le focus entre les boutons. RETURN, ENTER ou ESPACE appuient sur le bouton qui a le focus |

F1–F6 sont les touches f1–f6 du pavé numérique du CPC.

## Le pointeur et la souris

- **Avec une souris AMX :** le pointeur suit la souris, et son bouton sert à cliquer. La souris s'active et se règle dans les Réglages (SOURIS AMX, LECTURE AMX).
- **Sans souris :** le pointeur se déplace avec les flèches, et COPY sert à cliquer. Dans les vues qui utilisent les flèches pour choisir (Bureau, Lanceur, Commander), les flèches choisissent les fichiers et les programmes au lieu de déplacer le pointeur.

La plupart des choses fonctionnent des deux façons : un clic sur un fichier ou un programme le sélectionne, un second clic l'ouvre.

## Fenêtres et dialogues

- **Déplacement :** toute fenêtre peut être déplacée par sa barre de titre : les dialogues, la fenêtre FICHIERS du Bureau, l'accessoire de bureau et les fenêtres des programmes en blocs. Tant que vous tenez le bouton, un contour en pointillés s'affiche, et la fenêtre est redessinée là où vous le relâchez.
- **Dialogues :** ils attendent une réponse. TAB et ←/→ changent de bouton, RETURN l'actionne, et ESC choisit le dernier (en général NON ou ANNULER).
- **Saisie de texte** (par ex. pour renommer) : vous tapez à la fin, DEL efface, RETURN valide et ESC annule. Vous pouvez aussi taper dans la langue du module.

## Disquettes et fichiers

- **Format :** V-OS lit et écrit les disquettes AMSDOS (format DATA ou SYSTEM) dans A: et B:. Le BASIC peut lui aussi lire ses fichiers.
- **Noms :** 8 caractères plus 3 pour le type (`NOM.TYP`).

Les types que vous rencontrerez :

| Type | Ce que c'est |
|---|---|
| `.BIN` | un programme V-OS (ou un fichier binaire) |
| `.VBN` | un programme V-OS fait avec des blocs |
| `.VBS` | un programme Studio (sa source) |
| `.OVL`, `.MSG`, `.LNG` | des parties de V-OS lui-même : vues, textes, langues |
| `.SYS` | des fichiers système (par ex. `VBRUN.SYS`, le lecteur de Studio) |
| `SETUP.CFG` | les réglages |

Les réglages, les langues et les vues sont lus depuis A:, la disquette de V-OS doit donc y rester.

## Messages d'erreur

Les erreurs de disquette s'affichent dans un dialogue : LECTEUR PAS PRÊT, PROTÉGÉ EN ÉCRITURE,

FICHIER INTROUVABLE, DISQUE PLEIN, RÉPERTOIRE PLEIN, NOM DÉJÀ PRIS, NOM INVALIDE et d'autres.

Deux autres concernent les programmes :

- **PROGRAMME NON V-OS :** le fichier n'a pas l'en-tête d'un programme.
- **FERMEZ D'ABORD UN AUTRE PROGRAMME :** il n'y a plus de place. Voir le chapitre 3.

# 2. Les vues

V-OS a quatre « vues » : des façons de voir les fichiers et les programmes, et les réglages. On change de vue avec F1–F4 ou depuis le menu AFFICHAGE. Chaque vue retient le fichier sélectionné, et la suivante sélectionne le même.

## Les menus communs

Les deux derniers menus sont les mêmes dans toutes les vues et dans les programmes :

| **AFFICHAGE** | | **AIDE** | |
|---|---|---|---|
| BUREAU | F1 | TOUCHES | ^K |
| PROGRAMMES (le Lanceur) | F2 | À PROPOS DE V-OS | |
| COMMANDER | F3 | | |
| RÉGLAGES | F4 | | |
| PROGRAMMES (en cours) | F6 | | |
| CHANGER DE LANGUE | ^L | | |

## Bureau, F1

Mode 1. Une fenêtre FICHIERS montre les fichiers d'un lecteur sous forme d'icônes, quatre par rangée. Le type est écrit dans l'icône et le nom en dessous, et un `.VBN` porte une marque « lecture » à côté. À droite, deux icônes choisissent A: ou B:. La barre de titre indique le format de la disquette et le nombre de fichiers, et l'espace libre s'affiche en bas.

| **Touche** | **Ce qu'elle fait** |
|---|---|
| flèches | choisir un fichier (↑/↓ une rangée) |
| TAB | l'autre lecteur |
| RETURN, ENTER | infos du fichier |
| ^C | copier sur l'autre lecteur |
| ^D | supprimer (demande d'abord) |
| ^N | renommer |
| ^R | relire les disques |
| souris | clic sur un fichier : sélection, encore : infos. Clic sur un lecteur : l'ouvre |

- **Menu FICHIER :** LANCER (lance le programme sélectionné), INFOS DU FICHIER, COPIER SUR L'AUTRE LECTEUR, SUPPRIMER, RENOMMER, RELIRE LES DISQUES, QUITTER VERS LE BASIC.
- **La fenêtre FICHIERS** se déplace par son titre et reste là où vous la mettez, même quand vous changez de vue.

## Lanceur (PROGRAMMES), F2

Mode 0, 16 couleurs. Il ne montre que les programmes de A: (`.BIN`, `.VBN`, `.BAS`), sous forme de tuiles dans une grille de 3×3, avec les pages suivantes en dessous. Chaque programme qui a une icône l'affiche sur sa tuile. La tuile sélectionnée est blanche.

| **Touche** | **Ce qu'elle fait** |
|---|---|
| flèches | choisir (↑/↓ une rangée. Au-delà du bord, la page suivante ou précédente) |
| RETURN, ENTER | lance le programme. S'il tourne déjà, il reprend là où il en était |

| | |
|---|---|
| ^A | le charge comme accessoire de bureau (COMME ACCESSOIRE). F5 l'ouvre ensuite |
| ^R | relire la disquette |
| souris | clic : sélection, encore : lancement |

**Menu FICHIER :** LANCER, COMME ACCESSOIRE, RELIRE LES DISQUES, QUITTER VERS LE BASIC.

## Commander, F3

Mode 1, deux panneaux avec des listes de fichiers : A: à gauche, B: à droite. Chaque panneau indique le format et le nombre de fichiers dans son titre, et l'espace libre en bas. Le panneau actif a un titre foncé et une sélection.

| Touche | Ce qu'elle fait |
|---|---|
| ↑/↓ | choisir (avec SHIFT : une page) |
| TAB, ←/→ | l'autre panneau |
| RETURN, ENTER | infos : taille, type, adresse de chargement, les premiers octets |
| ^C | copier vers l'autre panneau, quelle que soit la taille. S'il existe un fichier du même nom, demande d'abord |
| ^D | supprimer |
| ^N | renommer |
| ^R | relire les deux disquettes |
| souris | clic : sélection, encore : infos |

**Menu FICHIER :** LANCER, INFOS DU FICHIER, COPIER EN FACE, SUPPRIMER, RENOMMER, RELIRE LES DISQUES, FORMATER LE DISQUE (DATA), QUITTER VERS LE BASIC.

> FORMATER efface tous les fichiers de la disquette du panneau actif. Une confirmation est demandée.

## Réglages, F4

Mode 1. Une ligne par réglage. Les changements prennent effet tout de suite. Une fois enregistrés (RETURN ou ^S), ils sont écrits dans `A:SETUP.CFG` et s'appliquent aussi au démarrage suivant.

| Réglage | Valeurs |
|---|---|
| VUE AU DÉMARRAGE | BUREAU, PROGRAMMES, COMMANDER |
| LANGUE | ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO |
| DÉLAI RÉPÉTITION | combien de temps on tient une touche avant qu'elle se répète : COURT, MOYEN, LONG |
| VITESSE RÉPÉTITION | LENTE, MOYEN, RAPIDE |
| SOURIS AMX | OUI, NON |
| LECTURE AMX | IMPULSIONS ou NIVEAUX : la façon de lire la souris. Si le pointeur ne bouge pas correctement, essayez l'autre |
| COULEUR DU BUREAU | BLEU, NOIR, VERT, ROUGE |

| Touche | Ce qu'elle fait |
|---|---|
| ↑/↓ | choisir un réglage |

| | |
|---|---|
| ←/→ | changer la valeur (un clic sur la ligne aussi) |
| RETURN, ^S | enregistrer |

Changer la LANGUE charge aussitôt le module de langue depuis A:. Voir le chapitre 4.

# 3. Les programmes

## Lancer et quitter

Un programme se lance de l'une de ces façons :

- depuis le Lanceur : sélectionnez-le et appuyez sur RETURN, ou cliquez deux fois
- depuis le Bureau ou le Commander : FICHIER > LANCER
- depuis un autre programme, comme Studio, qui lance ce que vous créez

Chaque programme a son propre menu à côté des menus communs AFFICHAGE et AIDE. Il se termine depuis son menu (en général BACK ou STOP, CONTROL+Q) et revient à la vue d'où il a été lancé.

## Plusieurs programmes à la fois

Les programmes V-OS sont de deux sortes :

- **Petits, avec une table de relocation :** DEMO, MIXED et CLOCK, les exemples de l'éditeur web (PONG, CATCH, STARCAT, COUNTER), et tout ce que produit le SDK. Ils se partagent 12K de mémoire, et tous ceux qui y tiennent tournent ensemble.
- **Programmes qui demandent toute la mémoire :** STUDIO et tout ce que fait Studio (CALC, MINES, FLAP, TICKER et les vôtres). Sans extension mémoire, ils tournent seuls. Si l'un d'eux tourne déjà, vous obtenez « FERMEZ D'ABORD UN AUTRE PROGRAMME ».

Quand vous passez à une vue (F1–F4) pendant qu'un programme tourne :

- Un petit programme reste en mémoire. Vous le retrouvez depuis le Lanceur (RETURN dessus) ou avec F6, et il reprend exactement là où il en était.
- Un programme qui occupe toute la mémoire se termine, sauf s'il y a une extension mémoire.

**F6** (AFFICHAGE > PROGRAMMES) montre la liste des programmes en cours. On en choisit un avec les flèches et RETURN. Si aucun ne tourne, vous obtenez « AUCUN PROGRAMME EN COURS ».

**Avec une extension mémoire** (par ex. 512K, que V-OS détecte tout seul au démarrage), chaque programme qui demande toute la mémoire reçoit sa propre banque. Ainsi STUDIO, CALC, DEMO et PONG peuvent tous tourner ensemble.

## En arrière-plan

Un programme qui se trouve derrière un autre, ou derrière une vue, est normalement **figé** et reprend quand il revient au premier plan.

Certains programmes demandent à **continuer à tourner** en arrière-plan, par ex. TICKER, qui compte le temps. Pendant ce temps, ils ne dessinent pas et ne jouent pas de son. Quand ils reviennent au premier plan, ils ont aussi compté le temps de leur absence.

Dans Studio, vous choisissez cela pour votre programme avec STUDIO > IN BACKGROUND (en arrière-plan).

## L'accessoire de bureau (ACCESSOIRE)

Un petit programme, comme CLOCK, peut s'ouvrir dans une fenêtre **par-dessus tout le reste** :

une vue ou un programme.

1. Dans le Lanceur, sélectionnez CLOCK et choisissez FICHIER > COMME ACCESSOIRE (^A).
2. Dès lors, **F5** l'ouvre et le ferme, où que vous soyez. ESC le ferme aussi.

Tant qu'il est ouvert, il reçoit les touches et les clics. Sa fenêtre se déplace par son titre. Quand il se ferme, ce qui était dessous est redessiné.

## Les programmes de la disquette

| Programme | Ce que c'est |
|---|---|
| **STUDIO** | l'éditeur de blocs : vous créez des programmes sur le CPC (chapitre 5) |
| **DEMO** | le premier programme V-OS : un carré qui se déplace tout seul. Les flèches le dirigent et ESPACE change sa couleur |
| **MIXED** | l'« écran mixte » : une bande en 16 couleurs (mode 0) entre des bandes en mode 1. Un clic affiche les coordonnées |
| **CLOCK** | le temps écoulé depuis le démarrage de V-OS. Il tourne aussi comme accessoire de bureau |
| **CALC** | une calculatrice dans une fenêtre avec des boutons, faite dans Studio (il existe aussi `CALC.VBS`) |
| **MINES** | un démineur de 8×6 avec 8 mines, fait dans Studio. Un clic ouvre une case, et NEW commence une nouvelle partie |
| **BRICKS** | un casse-briques : une raquette avec ←/→, 32 briques, 3 vies, des niveaux avec une balle toujours plus rapide. À GAME OVER, ESPACE pour une nouvelle partie |
| **VROUVES** | tower defence, d'après le jeu CPC LET'S GO FOR VROUVES : 7 niveaux, du Kafenio au cimetière, avec 10 vagues chacun. La disquette de V-OS a le niveau 1. Tous sont sur sa propre disquette, `vrouves.dsk` : en B:, et FICHIER > LANCER dans le Commander. Les flèches déplacent le curseur, 1–6 choisit un défenseur, ESPACE construit, S vend, N appelle la vague suivante, P met en pause. Écrit avec le SDK (`myapps/vrouves`) |
| **PONG** | une balle et une raquette avec les flèches, score et vies |
| **CATCH** | attrapez les étoiles qui tombent |
| **STARCAT** | un chat avec des costumes qui marche, une étoile, une mélodie tirée d'une liste |
| **COUNTER** | un compteur avec des boutons +1, −1 et 0, en mode 1 |
| **FLAP** | un oiseau qui bat des ailes (costumes), et une étoile qui change de costume chaque seconde |
| **TICKER** | compte les secondes et continue en arrière-plan |

- Les programmes en blocs sont des fichiers `.VBN`. Ils ont un menu PROGRAM > STOP (CONTROL+Q).
- Ceux qui ont aussi un `.VBS` (CALC, MINES, FLAP, TICKER) s'ouvrent dans Studio pour que vous puissiez les regarder ou les modifier.
- `VBRUN.SYS` n'est pas un programme : c'est le « lecteur » dont Studio a besoin.

# 4. Les langues

V-OS parle six langues. L'anglais est intégré. Les cinq autres sont des **modules de langue**, des fichiers sur la disquette qui apportent trois choses : tous les textes du système, le clavier et les lettres (la police) de la langue.

| Langue | Fichier | Clavier |
|---|---|---|
| English | intégré (`ENGLISH.MSG`) | celui, anglais, du CPC |
| Ελληνικά | `GREEK.LNG` | grec, avec accents |
| Français | `FRENCH.LNG` | AZERTY, chiffres avec SHIFT, touches mortes ^ et ¨ |
| Español | `SPANISH.LNG` | Ñ sur la touche `:`, touches mortes ´ ¨ ` ^, et ¡ ¿ |
| Deutsch | `GERMAN.LNG` | QWERTZ, Ü Ö Ä ß, touches mortes ´ et ` |
| Italiano | `ITALIAN.LNG` | è é ò à ù ì sur leurs propres touches |

## Changer de langue

- **Durablement :** dans les Réglages (F4), la ligne LANGUE fait défiler toutes les langues. Chacune se charge aussitôt depuis A:, et tout l'écran est redessiné dans cette langue. Enregistrer (^S) la conserve aussi pour le démarrage suivant.
- **Rapidement :** CONTROL+L (AFFICHAGE > CHANGER DE LANGUE) bascule entre l'anglais et la langue du module chargé.
- **Clavier seul :** CONTROL+SHIFT bascule le clavier entre l'anglais et celui du module. La barre de menus indique à droite lequel est actif (EN, ΕΛ, FR, ...).

Les symboles `@ # [ ] { } \ |` ne sont pas sur les claviers des modules, car le CPC n'a pas de touche AltGr. Tapez-les avec le clavier anglais.

## Les textes des programmes

Les programmes ont leurs propres textes en **deux langues** : l'anglais et, en option, une autre (aujourd'hui, le grec). Un programme n'apparaît dans sa seconde langue que si le module est dans cette langue. Sinon, il apparaît en anglais. Ainsi, en français, Studio apparaît en anglais.

## Créer une nouvelle langue

Les sources des langues se trouvent dans `lang/<code>/` :

- `messages.txt` : les textes du système. La liste de base est définie par `lang/en/messages.txt`, et tout ce qui manque dans un module apparaît en anglais.
- `keymap.txt` : les touches qui diffèrent du clavier anglais.
- `pack.txt` : le code, le nom, le jeu de caractères et la police.

`tools/mklng.py` les transforme en `.LNG`, et le Makefile le place sur la disquette. Les modules latins partagent la police `lang/latin1/font.txt`.

# 5. Studio : des programmes avec des blocs

STUDIO est un éditeur de blocs à la Scratch qui tourne sur le CPC lui-même.

- Vous créez des sprites et des scripts à partir de blocs.
- Le programme se lance aussitôt avec CONTROL+R.
- Il devient un programme V-OS normal (`.VBN`) qui se lance aussi depuis le Lanceur.

## L'écran

Studio utilise un écran « partagé », en deux zones :

- **En haut, en mode 0 (16 couleurs) :**
  - **à gauche, la scène** à demi-taille
  - **à droite, le panneau SPRITES :** les sprites avec leur couleur et leur position, la taille du sprite sélectionné (W, H), le fond et le mode du programme (par ex. `STAGE 2 M1`), et le nom du fichier
- **En bas, en mode 1 :**
  - **à gauche, la palette** avec les blocs d'une catégorie (`< EVENTS >`)
  - **à droite, le script**, une ligne par bloc
  - **tout en bas, la ligne d'état**, avec les touches qui fonctionnent là où vous êtes

**TAB** passe du script à la palette, puis à la scène, et revient au script.

## Les menus

| STUDIO | | EDIT |
|---|---|---|
| NEW | ^N | DELETE LINE |
| OPEN (`.VBS`) | ^O | + SPRITE (jusqu'à 6) |
| SAVE | ^S | − SPRITE |
| SAVE AS | | BACKGROUND (la couleur de la scène) |
| RUN | ^R | SCREEN MODE (0 ou 1) |
| IN BACKGROUND | | |
| BACK | ^Q | |

- **RUN (^R) :** enregistre `NOM.VBS`, écrit `NOM.VBN` et le lance. Le programme s'arrête depuis son propre menu (PROGRAM > STOP, ^Q), puis Studio redémarre avec le même programme ouvert.
- **IN BACKGROUND (en arrière-plan) :** demande si le programme continue à tourner quand il est derrière un autre ou derrière une vue, ou s'il se met en pause, ce qui est le comportement par défaut.
- **SCREEN MODE (mode d'écran) :**
  - **Mode 0 :** 16 couleurs, 192 pixels de large.
  - **Mode 1 :** 4 couleurs (encres 0–3), 384 pixels de large, et plus de texte.

## La palette

←/→ changent de catégorie, ↑/↓ choisissent un bloc, et RETURN l'ajoute sous la ligne du script. Un clic sur un bloc de la palette fait de même.

Un bloc qui s'ouvre (FOREVER, REPEAT, IF) entre avec son END, et s'efface avec lui. Un bloc avec du texte demande le texte tout de suite.

## Le script

| Touche | Ce qu'elle fait |
|---|---|
| ↑/↓ | ligne |
| ←/→ | case de la ligne (les cases sont les valeurs à l'intérieur du bloc) |
| RETURN | sans case sélectionnée : ajoute le bloc de la palette. Sur une case de texte : changer le texte. Sur une case de touche : attend que vous appuyiez sur la touche |
| CLR | efface la ligne |
| DEL | efface le dernier chiffre d'un nombre |
| souris | clic sur une case : la sélectionne. Second clic : change sa sorte |

Le programme est une liste de lignes :

- Chaque `WHEN …` commence un script, pour un sprite ou pour la scène. Le script va jusqu'au `WHEN` suivant.
- L'indentation se fait toute seule, d'après les blocs qui s'ouvrent et se ferment.
- Les lignes avant le premier `WHEN` sont ignorées.
- Tout ce qui reste ouvert est fermé à la fin du script.

### Ce qui va dans une case

La ligne d'état dit toujours ce que la case accepte.

**Case de valeur :**

| Touche | Valeur |
|---|---|
| 0–9, − | nombre, un entier de −32768 à 32767 |
| A–H | variable. La même lettre encore : A1, A2 … A15, puis retour à A |
| X, Y | la position du sprite du script |
| R et des chiffres | RANDOM : un nombre de 1 à n |
| K et une touche | KEY : 1 tant qu'elle est enfoncée, sinon 0 |
| T et un chiffre | TOUCHING : 1 quand le sprite touche le sprite n |
| Z | AT EDGE : 1 quand le sprite est au bord de la scène |
| P | BUTTON : le numéro du dernier bouton actionné |
| U | COSTUME : le numéro du costume du sprite (0 : aucun costume) |

**Autres sortes de case :**

- **Variable :** A–H, et la même lettre encore pour A1…A15.
- **Comparaison :** `<`, `=`, `>`. ESPACE donne « aucune », et alors IF vérifie seulement que la première valeur n'est pas 0.
- **Sprite** (dans les blocs `WHEN`) : 0 la scène, 1–6 un sprite.
- **Touche** (dans `WHEN KEY`) : RETURN puis la touche.
- **Texte :** RETURN, et vous tapez jusqu'à 14 caractères.

- **Groupe de variables** (SAVE/LOAD VARS) : A–H passe au groupe de huit suivant (A-H, A1-H1 … A15-H15), et ESPACE donne ALL.

Les lettres sont lues comme des touches, elles fonctionnent donc aussi avec le clavier grec.

## La scène et les sprites

| Touche | Ce qu'elle fait |
|---|---|
| flèches | déplacent le sprite sélectionné |
| 1–6 | sélectionner un sprite |
| C | couleur suivante |
| W, H | largeur, hauteur |
| K | le costume qu'il porte (aucun, 1, 2, …) |
| E | dessiner son costume (un nouveau, s'il n'en a pas) |
| clic sur la scène | place le sprite à cet endroit |

Un sprite sans costume est un rectangle de couleur.

### Les costumes

**E** ouvre l'éditeur de costumes à la place de la scène. Il a une grille de 16×16 avec les 16 encres en dessous.

| Touche | Ce qu'elle fait |
|---|---|
| flèches | case |
| ESPACE, RETURN | peint la case |
| 0–9, A–F | encre |
| . | transparent |
| E, TAB | terminé |
| clic | sur une case : la peint. Sur la rangée d'encres : choisit une encre |

Un programme a jusqu'à 8 costumes, partagés par tous les sprites. Un sprite avec costume mesure 16×16. Sur la scène, x est arrondi à un octet entier (2 pixels en mode 0, 4 en mode 1).

## Tous les blocs

Dans la colonne « Bloc », les cases apparaissent comme dans le script.

### EVENTS (événements)

| Bloc | Ce qu'il fait |
|---|---|
| WHEN START *sprite* | le script démarre avec le programme |
| WHEN KEY *touche sprite* | démarre quand on appuie sur la touche |
| WHEN CLICKED *sprite* | démarre sur un clic sur le sprite (ou sur la scène) |
| WHEN I RECEIVE *n sprite* | démarre quand le message n est envoyé |
| SEND *n* | envoie le message n (n peut aussi être une variable) |

> Un message relance **depuis le début** tout script qui l'attend, même s'il tournait déjà. C'est pourquoi un script ne devrait pas s'envoyer un message à lui-même.

### CONTROL (contrôle)

| Bloc | Ce qu'il fait |
|---|---|

| | |
|---|---|
| FOREVER … END | répète indéfiniment |
| REPEAT *n* … END | n fois |
| IF *a comparaison b* … [ELSE …] END | exécute les lignes seulement si la comparaison est vraie |
| WAIT *n* | attend n trames (50 = une seconde) |
| WAIT UNTIL *a comparaison b* | attend que la comparaison soit vraie |
| STOP | termine ce script |

## MOTION (mouvement)

| Bloc | Ce qu'il fait |
|---|---|
| GO TO *x y* | le sprite va en x, y |
| CHANGE X *n*, CHANGE Y *n* | déplace de n |
| SPEED *vx vy* | fixe sa vitesse |
| MOVE | un pas à sa vitesse |
| BOUNCE | au bord de la scène, inverse la vitesse |

## LOOKS/SOUND (apparence et son)

| Bloc | Ce qu'il fait |
|---|---|
| COLOUR *n* | l'encre du rectangle |
| COSTUME *n* | porte le costume n (1–8. 0 ne change rien) |
| NEXT COSTUME | le suivant, et après le dernier, de nouveau le premier |
| SHOW, HIDE | |
| PRINT *valeur* AT *x y* | affiche un nombre |
| NOTE *n* FOR *trames* | joue une note (numéro MIDI : 60 = do, 69 = la 440 Hz) et attend qu'elle se termine |

## VARIABLES

| Bloc | Ce qu'il fait |
|---|---|
| SET *variable* TO *valeur* | |
| CHANGE *variable* BY *valeur* | addition |
| DECREASE *variable* BY *valeur* | soustraction |
| MULTIPLY *variable* BY *valeur* | |
| DIVIDE *variable* BY *valeur* | division entière |
| REMAINDER *variable* BY *valeur* | le reste de la division |
| SAVE VARS *groupe* TO *"nom"* | écrit les variables dans un fichier texte (voir plus bas) |
| LOAD VARS *groupe* FROM *"nom"* | les relit |

## WINDOWS (fenêtres)

| Bloc | Ce qu'il fait |
|---|---|
| WINDOW *"titre" largeur hauteur* | une fenêtre au milieu de la scène, comme les dialogues de V-OS. Une nouvelle fenêtre remplace la précédente et ses boutons |
| BUTTON *"texte"* AT *x y* | un bouton du système. Il s'actionne d'un clic ou avec TAB et RETURN. Les boutons sont numérotés à partir de 0, dans l'ordre où ils apparaissent |
| WHEN BUTTON *sprite* | démarre quand un bouton quelconque est actionné. La valeur BUTTON (P) dit lequel |
| TEXT *"texte"* AT *x y* | écrit du texte |

| | |
|---|---|
| LABEL *bouton* TO *"texte"* | change le texte d'un bouton |

Avec une fenêtre ouverte, TEXT, BUTTON et PRINT comptent à partir du coin supérieur gauche de son intérieur. La fenêtre se déplace par son titre, avec ses boutons.

## Les variables

Il y a 128 variables, partagées par tous les scripts, nommées A–H, A1–H1, …, A15–H15. Au départ, elles valent toutes 0, et elles contiennent des entiers de −32768 à 32767.

### Enregistrer et restaurer (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` écrit les 128 dans le fichier `GAME.TXT`. Le `.TXT` est ajouté quand le nom n'a pas de type.
- Avec un groupe (`A-H`, `A1-H1`, …), seules ces 8 sont écrites.
- Le fichier est un texte, une ligne par variable :

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` modifie les variables que le fichier mentionne. Avec un groupe, il ne modifie que celles qui sont aussi dans le groupe. Les lignes qu'il ne comprend pas sont ignorées, et si le fichier manque, rien ne change.
- Un fichier peut donc restaurer toutes les variables ou un seul groupe de huit, par ex. les positions sauvegardées d'un jeu.

## Les fichiers

- **`NOM.VBS` :** le programme tel que vous l'éditez. OPEN l'ouvre, et jusqu'à 5K sont lus.
- **`NOM.VBN` :** le programme prêt à tourner. RUN l'écrit, et il se lance aussi depuis le Lanceur. C'est le lecteur (`VBRUN.SYS`) avec le programme. Le lecteur compile les blocs en code Z80 au démarrage, si bien que les programmes tournent vite.

Un `.VBS` peut aussi être fabriqué à partir de texte sur le PC, avec `tools/mkvbs.py`. La syntaxe est décrite en tête du fichier, et les exemples sont `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS).

## Les limites

| | |
|---|---|
| Lignes (blocs) | 240 |
| Sprites | 6 |
| Scripts (`WHEN …`) | 16 |
| Variables | 128 |
| REPEAT dans un programme | 64 (au-delà, ils partagent un compteur) |
| Blocs ouverts imbriqués les uns dans les autres | 16 |
| Textes | 32, jusqu'à 14 caractères chacun |
| Costumes | 8, de 16×16 |
| Boutons | 64 |
| Code | au moins 4 000 octets, même pour le plus gros programme. Ce qui ne tient pas est ignoré |

## Conseils

- **Scripts de la scène :** les blocs de mouvement et d'apparence ne font rien, et X, Y, TOUCHING et AT EDGE donnent 0.
- **TOUCHING :** n'est vrai que si les deux sprites sont visibles.
- **Synchronisation :** les scripts tournent « ensemble ». Chaque script cède son tour à la fin de chaque boucle et à chaque WAIT. Pour qu'un script en attende un autre, utilisez une variable et `WAIT UNTIL` (voir MINES).
- **Exemples tout prêts :** CALC, MINES, FLAP et TICKER s'ouvrent dans Studio (OPEN).

# 6. Les blocs sur le PC : l'éditeur web et `vbc.py`

En plus de Studio, qui tourne sur le CPC, il existe aussi un éditeur de blocs pour le PC. Le langage est le même, avec quelques blocs en plus : listes, bruit, taille des sprites, texte en couleur.

## L'éditeur

Ouvrez `blocks/editor.html` dans un navigateur. Il est basé sur Blockly et existe en grec ou en anglais.

- **Catégories :** événements, contrôle, mouvement, apparence, texte, opérateurs, capteurs, variables, listes et fenêtres.
- **Sprites :** rectangles ou costumes. Ils sont définis dans un tableau, et la scène les montre dans les couleurs du CPC. On peut les déplacer à la souris.
- **Fichiers :** il ouvre et enregistre des fichiers `.vbp`. Ce sont des fichiers JSON contenant tout le programme, ils peuvent donc aussi s'écrire à la main.

## Compilation

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` écrit `build/NOM.ASM`, avec un commentaire pour chaque bloc. Il l'assemble avec RASM en `NOM.VBN`, avec une table de relocation pour qu'il tourne à côté d'autres programmes, et avec `--dsk` il le place sur la disquette.

| Option | Ce qu'elle fait |
|---|---|
| `-o build` | dossier de sortie |
| `--asm-only` | seulement le `.ASM` |
| `--dsk build/vos.dsk` | aussi sur la disquette |

## Le format `.vbp`

La description complète se trouve en tête de `blocks/vbc.py` : événements, commandes, valeurs, costumes, icône. En bref :

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variables et listes :** elles ont des noms, pas des lettres. Les listes ont une longueur maximale fixe.
- **Icône :** `"icon"` donne une icône pour le Lanceur.
- **Un programme avec beaucoup de parties semblables :** un script Python peut l'écrire. Voir `tools/mkbricks.py`, qui écrit `bricks.vbp` avec 32 briques.

## Exemples

Dans `blocks/examples/` :

| Fichier | Programme |
|---|---|
| `pong.vbp` | PONG |
| `catch.vbp` | CATCH |
| `starcat.vbp` | STARCAT : costumes, un air tiré d'une liste |
| `counter.vbp` | COUNTER : une fenêtre et des boutons |
| `bricks.vbp` | BRICKS (écrit par `tools/mkbricks.py`) |

Les fichiers `*.vbt` sont des programmes Studio sous forme de texte, pour `tools/mkvbs.py`. Voir le chapitre 5.

## Studio ou éditeur web ?

| | Studio | éditeur web |
|---|---|---|
| Où | sur le CPC | sur le PC |
| Fichier | `.VBS` | `.vbp` |
| Variables | 128, A–H15 | avec des noms |
| Listes, bruit, taille des sprites | non | oui |
| SAVE/LOAD VARS | oui | non |
| Le programme | demande toute la mémoire | petit, tourne à côté d'autres |

# 7. Le SDK de V-OS : des programmes en assembleur

Avec ce SDK, vous écrivez vos propres programmes en assembleur Z80. Ils tournent dans V-OS comme DEMO ou CLOCK : ils apparaissent dans le Lanceur, le Bureau et le Commander, et ils ont leur propre menu, des boutons, des dialogues, du son et des fichiers. Ils tournent aussi à côté d'autres programmes.

## Démarrage rapide

```
make new-app NAME=hello
```

Cela crée le dossier `myapps/hello/` à partir de `sdk/template/`. Il contient trois fichiers :

- `hello.asm` : le programme. Les flèches déplacent une boîte, il y a un bouton BEEP, un compteur de secondes, et son propre menu (ABOUT, BACK) à côté d'AFFICHAGE et AIDE.
- `texts.txt` : ses textes, en anglais et en grec.
- `icon.txt` : son icône, 16×16, pour le Lanceur.

```
make app NAME=hello
```

Cela construit `build/HELLO.BIN` et le place sur la disquette `build/vos.dsk`.

```
make run
```

Cela ouvre la disquette dans Caprice32. HELLO se trouve dans le Lanceur, à la dernière page.

- Un simple `make` construit tous les programmes de `myapps/` et les place sur la disquette, avec V-OS.
- Le nom a 1 à 8 lettres minuscules ou chiffres et commence par une lettre. Ce ne peut pas être le nom d'un programme de V-OS (par ex. demo).
- Il vous faut RASM, Python 3 et iDSK, comme pour tout V-OS.
- Dans Caprice32, vous lancez le programme vous-même, depuis le Lanceur.

Un programme plus grand :

- Mettez d'autres fichiers `.asm` dans son dossier (par ex. `art.asm` avec des sprites) et intégrez-les avec `include "art.asm"`. Le build les trouve, et reconstruit le programme quand ils changent.
- Ce que vous mettez dans le sous-dossier `disc/` (par ex. `disc/LEVEL1.DAT`) va tel quel sur la disquette, pour `VOS_LOAD`.
- Avec un sous-dossier `owndisc/`, le programme a aussi sa propre disquette, `build/<nom>.dsk`, avec le `.BIN` et les fichiers de `owndisc/`. Vous la mettez en B: et le lancez depuis le Commander (FICHIER > LANCER). Le lecteur d'où il a été lancé, pour y charger ses fichiers, est dans `VOS_SELF` : `db lecteur`, puis son nom (11 octets).
- Un programme avec une table de relocation (comme ceux du SDK) peut faire jusqu'à 12K. Pour le reste, chargez des fichiers dans la banque de données.

## Comment fonctionne un programme

Un programme est un `.BIN`, construit pour `VOS_APP_ADDR` (`#4000`). Il commence ainsi :

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
        jp init                 ; +0
        jp event                ; +3  HL -> l'événement
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 l'icône (facultative)
```

- **`init`** fixe le mode, la palette, le menu et les boutons, active `VE_FRAME` si besoin, et dessine tout.
  - A = 0 : un démarrage à neuf.
  - A = 2 : le programme revient d'une vue ou d'un autre programme. Son état est tel qu'il l'a laissé ; il refixe ce qui précède et dessine.
- **`event`** est appelé pour chaque événement et revient immédiatement. Un programme n'attend jamais de lui-même (pas de boucle d'attente). Pour le mouvement ou le temps, `VOS_TICKS` avec A = 1, et alors un `VE_FRAME` arrive 50 fois par seconde.
- **`redraw`** redessine tout, par ex. quand la langue change ou que l'accessoire de bureau se ferme.
- **Sortie :** `VOS_EXIT`, qui revient à la vue qui l'a lancé.

## L'ordre dans `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; les trames
        push hl
        call VOS_UI_EVENT               ; menus, boutons, CONTROL+lettre
        pop hl
        or a : jr nz,command            ; A = la commande
        call VOS_MENU_OPEN : ret nz     ; menu ouvert : les touches sont à lui
        ...                             ; vos propres touches et clics
command: ...
        jp VOS_COMMAND                  ; ce qui n'est pas à vous (AFFICHAGE, AIDE)
```

**Tant qu'un menu est ouvert, ne dessinez pas.** Sinon vous dessinerez par-dessus. Sur `VE_FRAME`, mettez l'état à jour, mais ne dessinez que si `VOS_MENU_OPEN` donne Z. Les dialogues (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) ont leur propre boucle, donc aucun événement n'arrive tant qu'ils sont ouverts.

## Les événements

`HL` → 8 octets : `db type, code, char, mods; dw x, y`.

| type | quoi | code / char / x, y |
|---|---|---|
| `VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2 | touche | code = numéro de touche (`VK_*`, rangée du clavier·8 + bit), char = le caractère dans la langue du module |
| `VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4 | pointeur | x, y en pixels du mode |
| `VE_FRAME` 8 | trame (avec `VOS_TICKS`) | — |

| | | |
|---|---|---|
| `VE_LANG` 7 | la langue du clavier a changé | — |

### Trois sortes de programme

- **Avec une table de relocation (le choix par défaut du SDK) :** `make app` le construit deux fois, en `#4000` et en `#4100`. `tools/mkreloc.py` ajoute la table d'après la différence.
  - Il tourne à côté d'autres programmes, dans les 12K à partir de `#5000`, et reprend là où il s'était arrêté (F6 montre les programmes en cours).
  - Ses données doivent être dans ses propres octets (étiquettes, `ds`), jamais à des adresses fixes.
  - Si `mkreloc.py` donne une erreur, une adresse est utilisée quelque part d'une façon qui ne survit pas à la relocation. Par ex. `ld a,label & 255`, ou seulement l'octet bas d'une adresse.
- **Sans table :** il prend toute la banque (jusqu'à 16K, code et données), mais ne tourne que seul, sauf s'il y a une extension mémoire. Il faut alors votre propre règle dans le Makefile (voir `STUDIO.BIN`).
- **Accessoire de bureau :** jusqu'à 4K, avec une table. Il s'ouvre avec F5 par-dessus tout le reste, et `init` reçoit A = 1. Voir `apps/clock/clock.asm` et la description dans `src/api/vos_api.inc`.

### Règles

- **Registres :** les appels altèrent tous les registres sauf IX. Les arguments vont dans A, BC, DE, HL (IX pour `VOS_RECT` et `VOS_BLIT`). Les résultats reviennent dans A ou HL. En cas d'échec, la retenue (carry) est mise et A = numéro d'erreur, que `VOS_ERROR` affiche.
- **Mémoire :** ne mettez pas une autre banque en `#4000`, car le programme disparaîtrait sous vos pieds. Pour 16K de plus, il y a la banque de données (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). Elle est partagée avec les vues et les autres programmes, et ne garde ses données que tant que le programme tourne.
- **Coordonnées :** x en pixels du mode (mode 0 : 0–191, mode 1 : 0–383), y de 0 à 258. La barre de menus occupe y 0–11.
- **RASM :** les étiquettes ne sont pas sensibles à la casse (`BOX` et `box` sont identiques). L'hexadécimal s'écrit avec `#` et non avec `&`.

## Les formats de données

- **Barre de menus** (`VOS_SET_BAR`) : `db count; dw menu, ...`. Les menus communs AFFICHAGE et AIDE viennent de `VOS_MENU_VIEW` et `VOS_MENU_HELP`.
- **Menu :** `db title, count` et pour chaque élément `db text, command, letter, 0`.
  - text : `VOS_SEPARATOR` pour une ligne
  - command : 1.., ce que renvoie `VOS_UI_EVENT`
  - letter : raccourci CONTROL+lettre, ou 0
- **Boutons** (`VOS_SET_WIDGETS`, puis `VOS_DRAW_WIDGETS`) : `db count`, et pour chaque bouton `dw x, y; db width in characters, text, command, 0`.
- **Dialogue** (`VOS_DIALOG` → A = bouton, 0 le premier, ESC donne le dernier) : `db title, text, number of buttons (1-3), button, ...`. Pour les boutons, `VM_OK`, `VM_YES` et `VM_NO`.

- **Textes** (`VOS_TEXTS`) : produits par `tools/mktexts.py` à partir de `texts.txt`, avec des lignes `NAME | English | Greek`.
  - Il donne `T_NAME` (`VT_FIRST` = `#C0` et au-delà) et la table `texts:`. Jusqu'à 64 textes.
  - Un `|` sans espaces autour commence une nouvelle ligne dans les dialogues.
  - Les textes vont partout où un « texte » est demandé (menus, boutons, dialogues). Avec `VOS_MSG`, vous obtenez leur adresse pour `VOS_PRINT`.
- **Palette** (`VOS_PALETTE`) : 17 couleurs du firmware (0–26) : 16 encres et la bordure. En mode 1, seules les encres 0–3 comptent.
- **Rectangle** (`VOS_RECT`, IX →) : `dw x, y, w, h; db pen`.
- **Icône :** 16 lignes de 16 caractères dans `icon.txt`. Chaque caractère est une encre en hexadécimal (mode 0, palette du Lanceur) ou `.` pour transparent.
- **Noms de fichiers** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`) : 11 octets, `"NAME EXT"`. Lecteur 0 = A:, 1 = B:.

## L'API (version 11)

| Appel | Entrée → sortie | Ce qu'il fait |
|---|---|---|
| `VOS_EXIT` | — | fin, retour à la vue qui l'a lancé |
| `VOS_TICKS` | A = 1 / 0 | `VE_FRAME` 50 fois par seconde, ou aucun |
| `VOS_FRAMES` | → HL (et A bits 16–23) | trames depuis le démarrage |
| `VOS_RANDOM` | → A | 0–255 |
| `VOS_COMMAND` | A = commande | les commandes communes d'AFFICHAGE et AIDE |
| `VOS_ERROR` | A = erreur | dialogue d'erreur |
| `VOS_MODE` | A = 0 / 1 | mode 0 (192×259, 16 encres) ou 1 (384×259, 4 encres) |
| `VOS_PALETTE` | HL → 17 couleurs | |
| `VOS_CLS` | A = encre | efface l'écran |
| `VOS_PLOT` | HL = x, DE = y, A = encre | un pixel |
| `VOS_RECT` | IX → rectangle | rectangle plein |
| `VOS_COLOURS` | A = encre du texte, C = fond | pour `VOS_PRINT` et `VOS_CHAR` |
| `VOS_PRINT` | HL = x, DE = y, BC → texte, 0 | |
| `VOS_CHAR` | HL = x, DE = y, A = caractère | |
| `VOS_MSG` | A = texte → HL | l'adresse d'un texte (`T_*` ou un texte du système) |
| `VOS_SET_BAR`, `VOS_DRAW_BAR` | HL → barre de menus | |
| `VOS_UI_EVENT` | (l'événement en cours) → A | commande de menu, de bouton ou de raccourci, 0 pour aucune |
| `VOS_DIALOG` | HL → dialogue → A | |
| `VOS_INPUT` | A = titre, HL → texte, C = maximum → carry si annulé | saisie de texte (demande C + 2 octets) |
| `VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS` | HL → boutons | |

| | | |
|---|---|---|
| VOS_MENU_VIEW, VOS_MENU_HELP | → HL | les menus communs |
| VOS_CATALOG | A = lecteur → A = fichiers, HL → liste | 16 octets chacun : 11 pour le nom, en +12 la taille en K |
| VOS_FILE_PICK | A = titre → A = fichier, carry si annulé | dialogue de sélection, après VOS_CATALOG |
| VOS_LOAD | A = lecteur, HL → nom, DE = où → BC = longueur | |
| VOS_SAVE | A = lecteur, HL → nom, DE = depuis, BC = longueur | fichier binaire, qui remplace l'ancien |
| VOS_DELETE | A = lecteur, HL → nom | |
| VOS_TEXTS | HL → table de textes | les textes du programme |
| VOS_MENU_OPEN | → NZ si un menu est ouvert | |
| VOS_KEY_HELD | A = touche → NZ tant qu'elle est enfoncée | pour les jeux |
| VOS_BLIT | IX → `dw x, y; db bytes, lines; dw source; db source stride` | image : paires (masque, pixels) |
| VOS_SOUND | A = canal 0–2, HL = période (bit 15 : bruit), C = volume 0–15, B = trames | une note ; 440 Hz correspond à la période 142 |
| VOS_SPLIT | HL → `dw y1, y2; db 8 colors` (0 = désactive) | écran mixte : mode 0 entre y1 et y2 |
| VOS_RUN | A = lecteur, HL → nom | lance un autre programme, puis celui-ci redémarre |
| VOS_KEEP | → HL → 16 octets | mémoire qui reste entre les programmes |
| VOS_BANK_GET, VOS_BANK_PUT | HL = depuis, DE = vers, BC = octets | copie depuis et vers la banque de données |
| VOS_BACKGROUND | A = 1 / 0 | continue à tourner ou se fige quand il est derrière un autre |
| VOS_DRAG | HL = x, DE = y, BC = w, A = h → carry : nouveaux HL, DE | déplacement d'une fenêtre par son titre |
| VOS_FIT | idem → HL, DE | ramène une fenêtre dans l'écran |

Tous les détails, par ex. sur l'arrière-plan, l'accessoire et `VOS_SPLIT`, sont dans `src/api/vos_api.inc`. Pour des exemples, voir `apps/demo/demo.asm` (mode 0, mouvement), `apps/clock/clock.asm` (accessoire) et `apps/mixed/mixed.asm` (écran mixte). Un jeu complet avec des sprites, un niveau lu dans un fichier et du son : `myapps/vrouves/`.

## Les tests

`make test-sdk` construit le modèle sous le nom TEMPLATE.BIN et l'ouvre depuis le Lanceur dans Caprice32, avec des appuis de touches définis par `src/kernel/test_script.asm` (script 15). Il prend des captures dans `build/shots` : son écran, la boîte après son déplacement, son menu, le dialogue ABOUT, et le retour au Lanceur.

Pour votre propre test automatique, copiez le script 15 et la cible `test-sdk` du Makefile.

# 8. Développer V-OS

Pour ceux qui travaillent sur V-OS lui-même. Les programmes *pour* V-OS sont décrits dans le SDK.

## Ce qu'il vous faut

- RASM, l'assembleur
- iDSK, pour les images de disquette
- Python 3, pour les outils. `make docs` demande aussi `markdown` et `pymupdf`.
- GNU make 4.3 ou plus récent
- pour les tests, Caprice32 (snap `caprice32`)

## Compilation

| Commande | Ce qu'elle fait |
|---|---|
| `make` | `build/vos.dsk` : le noyau, les vues, les langues, tous les programmes et `myapps/*` |
| `make run` | ouvre la disquette dans Caprice32 (`RUN"VOS"`) |
| `make shot` | démarre sans fenêtre et prend une capture d'écran |
| `make docs` | le manuel en PDF : `docs/V-OS.pdf` (grec) et `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it` |
| `make new-app NAME=x`, `make app NAME=x` | votre propre programme (SDK) |
| `make clean` | efface `build/` |

## Organisation du dépôt

| Dossier | Contenu |
|---|---|
| `src/vos.asm` | la racine de la compilation : chargeur et noyau, écriture sur la disquette |
| `src/boot/` | le chargeur qu'AMSDOS lance en `#4000`, détection de l'extension RAM |
| `src/kernel/` | vecteurs, interruptions, système de fichiers, vues, API (`api.asm`), réglages, extension du noyau (`ext.asm`), scripts de test (`test_script.asm`) |
| `src/drivers/` | Gate Array, CRTC, clavier, souris AMX, FDC, AY |
| `src/gfx/` | graphismes overscan, texte, écran mixte |
| `src/ui/` | menus, boutons, dialogues, saisie de texte, sélecteur de fichiers |
| `src/shell/` | les vues sous forme d'overlays : Bureau, Lanceur, Commander, Réglages |
| `src/api/vos_api.inc` | l'API pour les programmes |
| `lang/` | les langues (textes, claviers, polices) |
| `apps/` | les programmes en assembleur : DEMO, MIXED, STUDIO, CLOCK, et leurs icônes |
| `blocks/` | l'éditeur web, `vbc.py`, `runtime.asm`, le lecteur de Studio (`player.asm`), des exemples |
| `sdk/` | le SDK : guide et modèle de programme |
| `tools/` | outils : polices, langues, textes, icônes, relocation, `.VBS` à partir de texte, captures d'écran, PDF |

| | |
|---|---|
| `docs/` | ce manuel en grec (`manual/`) et en anglais, français, espagnol, allemand et italien (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), les PDF, les exigences (`REQUIREMENTS.md`) |

## La mémoire, en bref

| Zone | Ce qu'elle contient |
|---|---|
| `#0000-#3DFF` | le noyau. En page zéro, après les vecteurs de restart, la table de l'API en `#0108` |
| jusqu'à `#4000` | la pile du noyau |
| `#4000-#7FFF` | la « fenêtre » des banques |
| `#8000-#FFFF` | l'écran overscan de 32K. Dans ses trous, des tampons du noyau |

Les banques qui passent dans la fenêtre :

| Banque | Ce qu'elle contient |
|---|---|
| 4 | tampon de fichiers et de chargement, et la banque de données du programme en cours |
| 5 | la vue en `#4000-#4FFF`, et à partir de `#5000` les programmes avec table de relocation (ou un programme sans table dans toute la banque) |
| 6 | ce que cachent les menus et les dialogues, et à partir de `#7000` l'accessoire de bureau |
| 7 | le module de langue, les listes du Commander, les textes anglais, et à partir de `#7000` l'extension du noyau |

Avec une extension mémoire, ses banques accueillent des programmes. Les détails, adresse par adresse, sont dans `docs/REQUIREMENTS.md` (§4.2 et dans chaque section « Υλοποίηση M… » (réalisation)).

## Conventions

- **RASM :** l'hexadécimal s'écrit `#C000` et non `&C000`, car RASM lit `&` comme un ET. Les étiquettes ne sont pas sensibles à la casse.
- **Commentaires :** en anglais, courts, comme dans le code existant. Chaque fichier commence par un bloc qui dit ce qu'il fait.
- **Documentation :** en grec. `README.md` et `docs/REQUIREMENTS.md` ont une section par étape. Le manuel est écrit en grec et traduit dans les cinq autres langues de V-OS.
- **Textes :** les textes du système sont dans `lang/*/messages.txt`, et ceux des programmes dans `texts.txt` (pour `tools/mktexts.py`).

## Les tests

Les tests lancent V-OS dans Caprice32 sans fenêtre et prennent des captures d'écran dans `build/shots/<délai>/`.

**Comment ils fonctionnent :**

- Caprice32 sans fenêtre ne peut pas appuyer sur les flèches, COPY ou les touches f. C'est pourquoi les appuis de touches sont faits par une compilation spéciale : un `-DTEST_SCRIPT=n` place le script n dans `src/kernel/test_script.asm`, une table de touches par trame.
- Chaque cible `make test-…` construit une disquette avec son script et prend une capture à chaque délai de `…_SHOTS`.
- Le délai se mesure en appuis sur « Z » : environ 2,5–3,5 trames chacun. La correspondance n'est pas fixe, on saisit donc un moment avec deux ou trois délais autour.

- Dès que V-OS écrit sur la disquette, Caprice32 ne se ferme plus tout seul, et le test se termine par un dépassement de délai. C'est normal dans test-vars, test-costume et d'autres.
- Vous regardez les captures vous-même, par ex. réunies en une image avec `ffmpeg … hstack/vstack`.

| Cible | Ce qu'elle teste |
|---|---|
| `test-input` | touches, Studio : blocs, fichiers, exécution |
| `test-calc` | CALC avec des clics, puis `CALC.VBS` dans Studio |
| `test-costume` | costume dans Studio, exécution, réouverture |
| `test-acc` | l'horloge comme accessoire de bureau |
| `test-multi`, `test-switch`, `test-expand`, `test-behind` | plusieurs programmes, F6, extension mémoire, arrière-plan |
| `test-drag` | déplacement des fenêtres |
| `test-langs`, `test-lang` | les langues, l'enregistrement de la langue |
| `test-mines`, `test-bricks` | les jeux |
| `test-vars` | les 128 variables, SAVE/LOAD VARS |
| `test-sdk` | le modèle du SDK |
| `test-vrouves` | VROUVES depuis sa disquette en B:, par le Commander : construction, la vague 1, l'argent |
| `test-amsdos`, `test-reboot`, `test-system` | les fichiers V-OS en BASIC, redémarrage, disquette SYSTEM |

**Nouveau script :**

1. Ajoutez un `IF TEST_SCRIPT == n` au début de la chaîne dans `test_script.asm`, avec `ELSE` avant le suivant et un `ENDIF` de plus à la fin.
2. Ajoutez une cible au Makefile, comme `test-sdk`.

Chaque touche s'écrit `dw from, to : db row, bit` du clavier. Le numéro de touche vaut rangée·8 + bit, par ex. RETURN = 18 = rangée 2, bit 2.

**Sans émulateur :** le code d'un programme tourne aussi sur un Z80 dans Python (le paquet `z80`). Vous chargez le `.BIN` en `#4000` et faites revenir immédiatement les appels de l'API (ou leur faites faire ce qu'il faut, par ex. des fichiers dans un dictionnaire). Ensuite vous appelez init et event et lisez la mémoire. C'est bien plus rapide que les captures pour les erreurs de nombres, de fichiers et de sauts. Les adresses des étiquettes viennent de `rasm … -s -sl` (un fichier `.sym`).

## À chaque étape

1. Le code, et un test dans Caprice32 : nouveau ou existant, avec captures d'écran.
2. Les tests que le changement touche.
3. Une section dans `README.md` et une section « Υλοποίηση Μ… » (réalisation) dans `docs/REQUIREMENTS.md`.
4. **Mettre à jour le manuel dans les six langues** (`docs/manual/` en grec, `docs/manual/<code>/` pour en, fr, es, de, it, et `sdk/README.md`, `sdk/README.<code>.md` si l'API change), puis `make docs` pour tous les PDF.
5. Commit et push.